

Christian Bucher - Allgemeines technisches Portfolio

Gesamtprofil, Fähigkeitskarte und Evidenzroute

Portfoliothese

Meine Arbeit umfasst Agenteninfrastruktur, Memory- und Voice-Systeme, AMR-KI-Architektur, Projektplanung, IP-artige Formalisierung und adversariale Forschungsgrenzen.

Wie man dieses Portfolio liest

Starte mit den KI-Systemen, dann AMR/ECSA/FFG, dann IP-Formalisierung, dann Quantum/EXON als Red-Team-Reifefall.

Remote WhatsApp-to-Agent Command Loop

Ich wollte von unterwegs mit meinem Handy Arbeit an meinen Laptop übergeben koennen, ohne daraus eine gefaehrliche direkte Shell zu machen. Die WhatsApp-Nachricht sollte erst verstanden, in einen Arbeitsauftrag übersetzt, ausgeführt, bewertet und dann sauber zurückgemeldet werden.

- Das stärkste Signal ist nicht der Chatkanal, sondern die sichere Übersetzung von Alltagssprache in einen geprüften Arbeitsauftrag: Eingang, Interpretation, Ausführung, Bewertung und erst danach Rückmeldung.
- Persönlicher Prototyp bzw. entwickeltes System, keine Behauptung eines produktionsgehärteten Sicherheitsprodukts.

Was ich mir dabei gedacht habe

Was ich mir dabei gedacht habe, war einfach, aber technisch anspruchsvoll: Ich wollte nicht am Laptop sitzen müssen und trotzdem sinnvoll Arbeit an ihn übergeben können, ohne mein Handy zu einer blinden Remote-Shell zu machen. Der interessante Teil war die Vertrauensgrenze. Eine Nachricht musste erst zum Arbeitsauftrag werden, ausgeführt und danach bewertet werden, bevor sie zu mir zurückkommt.

Was ich gebaut habe

- Ich habe Fernzugriff als Autoritäts- und Sicherheitsproblem behandelt, nicht nur als Komfortfunktion.
- Ich habe die Front-Agenten-Schicht von der tool-reichen Ausführungsschicht getrennt.
- Ich habe mobile Sprache/Chat in strukturierte Arbeit mit Erfolgskriterien übersetzt.

Miguel Core Unified Daemon

Ich wollte meine vielen einzelnen Agenten-, Voice-, Memory- und Tool-Skripte nicht als lose Sammlung behalten. Miguel Core war mein Versuch, daraus eine beobachtbare lokale Kontrollschicht zu machen.

- Zeigt, dass ich viele einzelne KI-Experimente nicht nur gesammelt, sondern zu einer lokalen Kontrollschicht mit Routen, State, Health, Worker-Delegation und Grenzen verdichtet habe.
- Lokale experimentelle Kontrollschicht, nicht als fertiges kommerzielles Produkt formuliert.

Was ich mir dabei gedacht habe

Miguel Core entstand aus einer sehr praktischen Frustration: Ich hatte zu viele starke Teile als einzelne Skripte und Services. Ich wollte daraus eine lokale Kontrollschicht machen, die beobachtbar, routbar und verständlicher ist.

Was ich gebaut habe

- Ich denke Agenten nicht als einzelne Prompts, sondern als Betriebssystem-nahe Services.
- Ich kann verstreute Prototypen in eine zentrale Kontrollfläche zusammenführen.
- Ich habe früh über Health, State, Tool-Oberflächen und Autonomiegrenzen nachgedacht.

Multi-Model Cascade Router

Ich wollte nicht alles blind durch ein Modell schicken. Der Router behandelt Modelle als Ressourcen mit Stärken, Kosten, Quoten, Ausfällen, Qualität und Datenschutzkontext.

- Zeigt Modell-Governance in der Praxis: Modelle werden nach Kosten, Qualität, Privacy, Quoten, Ausfallrisiko und Eskalationslogik gewählt, nicht nach Bauchgefühl.
- Exakte Kostenersparnis nur dann behaupten, wenn neue Messlogs sie belegen.

Was ich mir dabei gedacht habe

Ich wollte Modellwahl nicht als Geschmack behandeln. Routing sollte Teil der Architektur sein: günstig, wenn möglich; stark, wenn nötig; lokal, wenn Privacy zählt; fallback-fähig, wenn ein Provider ausfällt.

Was ich gebaut habe

- Ich habe Modellwahl als Governance-System behandelt, nicht als Geschmacksfrage.
- Ich habe Kosten, Latenz, Quoten, Qualität und Fallbacks im selben System gedacht.
- Ich habe lokale und API-basierte Modelle in einer Routing-Logik verbunden.

Memory, Vault, CASS und Smart Memory MCP

Ich wollte Agenten nicht nur grössere Kontextfenster geben, sondern echtes Arbeitsgedächtnis: roh, zusammengefasst, prozedural, widerspruchsbewusst, veraltetheitsbewusst und wieder abrufbar.

- Zeigt Verständnis für Agenten-Zuverlässigkeit: Memory ist nicht nur Kontext, sondern Provenance, Widerspruchsbehandlung, Staleness, Zugriffshistorie und prozedurales Lernen.
- Praktische Memory-Infrastruktur, keine Behauptung menschenähnlichen Bewusstseins.

Was ich mir dabei gedacht habe

Die Memory-Arbeit entstand aus der Beobachtung, dass Agenten nicht nur mehr Kontext brauchen. Sie brauchen Memory-Hygiene: Rohfakten, Zusammenfassungen, prozedurales Lernen, Widersprüche, Veraltetheit und Provenance.

Was ich gebaut habe

- Ich habe Memory als Governance-Problem behandelt, nicht als Notizablage.
- Ich habe Rohdaten, Zusammenfassungen und Regeln getrennt.
- Ich habe Widerspruch, Veralterung und Herkunft als zentrale Risiken erkannt.

Voice Interface and Spoken Tool Use

Ich wollte mit meinem System sprechen koennen, aber Sprache ist für Tools riskant: sie ist schnell, ungenau und kann trotzdem Aktionen auslösen. Deshalb brauchte ich Muting, Tool-Grenzen, Fallbacks und Schutz gegen gefährliche Befehle.

- Zeigt eine sicherheitssensible Mensch-Maschine-Fläche: Voice macht Tool-Nutzung extrem direkt, deshalb müssen Mute, Bestätigung, Shell-Grenzen, Timeouts und Privacy mitgebaut werden.
- Terminal/Prototyp und Planungsstand sauber trennen; Wake-Word/macOS-Teile nur behaupten, wenn frisch getestet.

Was ich mir dabei gedacht habe

Voice war spannend, weil sich das System dadurch direkt anfühlte. Genau deshalb ist es aber riskant. Wenn ich spreche und der Computer handelt, dann sind Mute, Bestätigung, Shell-Grenzen und Privacy kein Zubehör, sondern Teil des Produkts.

Was ich gebaut habe

- Ich habe Voice als tool-fähige Kontrolloberfläche gebaut, nicht nur als Chat-Spielerei.
- Ich habe Datenschutz, Bestätigung und Unterbrechbarkeit von Anfang an mitgedacht.
- Ich habe hochwertige Stimmen mit praktischer Tool-Nutzung verbunden.